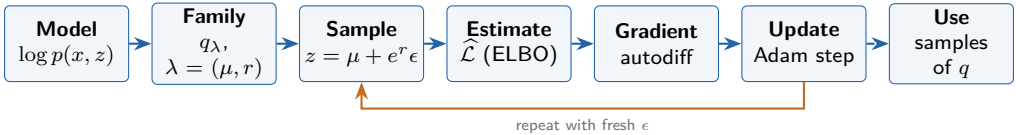
Variational inference by computation

From a noisy sensor to Bayesian neural networks

Miodrag Bolic

University of Ottawa
ELG 5218

Today: variational inference as a computation



- The previous lecture gave the *why*: ELBO, KL, mean-field, gradient estimators.
- Today is the *how*: the same seven steps run on a sensor, a regression and a small neural network.
- Only the **Model** box changes between problems; everything else is reused code.

By the end you can

write $\log p(x, z)$, fit q_λ by gradient steps, and answer questions with samples from q_λ.

Notebook §1

Running example: one sensor, one unknown

Noisy readings of a fixed quantity z:

$$x_n = z + \epsilon_n, \quad \epsilon_n \sim \mathcal{N}(0, \sigma_\epsilon^2),$$

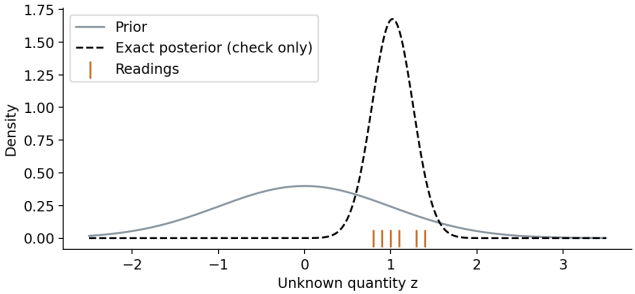
with $\sigma_\epsilon = 0.6$, prior $z \sim \mathcal{N}(0, 1)$ and readings

$$\mathbf{x} = (0.8, 1.1, 1.4, 0.9, 1.3, 1.0).$$

Questions:

- Which values of z are plausible?
- What is $P(z > 1 \mid \mathbf{x})$?
- What will the next reading be?

Notebook §1: data and model



The exact posterior (dashed) is known here only to *check* the algorithm; the optimizer never sees it.

What is fixed, what is random, what is learned

Quantity	Role	In code
$\mathbf{x}$, σ_ϵ, $p(z)$	data and model: fixed	`x_obs`, `noise_sd`, `prior`
$q_\lambda(z) = \mathcal{N}(\mu, \sigma^2)$	approximation we fit	`Normal(mu, log_sd.exp())`
$\lambda = (\mu, r)$, $\sigma = e^r$	**learned** by the optimizer	`mu`, `log_sd`
$\epsilon \sim \mathcal{N}(0, 1)$	**fresh** random draws each step	inside `rsample`
$z = \mu + \sigma\epsilon$	computed from both; gradients flow through it	`z`

- $r = \log \sigma$ is unconstrained, so the optimizer can never produce $\sigma \leq 0$.
- Two different SDs: σ_ϵ is sensor noise (model), σ is our uncertainty about z (learned).
- A single Gaussian has one peak: more iterations cannot fix a family that is too simple.

Pause and predict

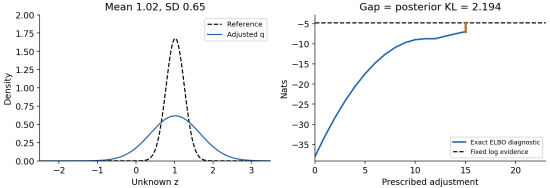
If we collect another reading, which of these change?

Notebook §1–§2

The objective: a score we can compute (recap)

$$\boxed{\log p(\mathbf{x}) = \mathcal{L}(\lambda) + \mathrm{KL}(q_\lambda \,\|\, p(z \mid \mathbf{x}))}, \qquad \mathcal{L}(\lambda) = \mathbb{E}_{q_\lambda}\Big[\underbrace{\log p(\mathbf{x} \mid z)}_{\text{fit}} + \underbrace{\log p(z)}_{\text{prior}} - \underbrace{\log q_\lambda(z)}_{\text{entropy}}\Big].$$

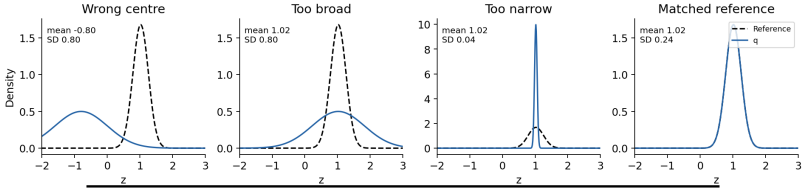
- Every term is computable at a sampled z; the unknown $p(\mathbf{x})$ is a constant.
- Higher ELBO = smaller posterior KL, by the same amount.
- Values are in nats, usually negative; only differences matter.



Orange gap = posterior KL as q is adjusted by hand.

Notebook §3: the bound and the gap

Reading ELBO values: the right centre is not enough



Approximation	Mean	SD	ELBO	Posterior KL
Wrong centre	-0.800	0.800	-38.076	33.265
Too broad	1.022	0.800	-8.751	3.941
Too narrow	1.022	0.040	-6.108	1.297
Matched reference	1.022	0.238	-4.811	0.000

Fixed $\log p(\mathbf{x}) = -4.811$ nats.

Pause and predict

Why is the narrowest Gaussian, with the tallest peak, not the best one?

Notebook §2–§3: candidates and exact diagnostics

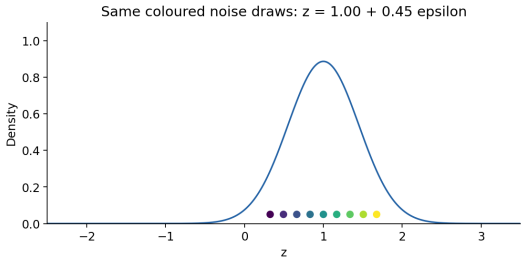
Monte Carlo estimate with reparameterized samples

Draw $\epsilon^{(s)} \sim \mathcal{N}(0,1)$ and set $z^{(s)} = \mu + \sigma\epsilon^{(s)}$:

$$\widehat{\mathcal{L}} = \frac{1}{S}\sum_{s=1}^{S}\Big[\sum_{n=1}^{N}\log p(x_n \mid z^{(s)}) + \log p(z^{(s)}) - \log q_\lambda(z^{(s)})\Big].$$

- Sum over observations, average over samples.
- For fixed ϵ, z is a differentiable function of (μ, r), so autodiff gives $\nabla_\lambda \widehat{\mathcal{L}}$.
- More samples $S \Rightarrow$ less Monte Carlo noise.

Notebook §3–§4



Same noise points: μ shifts them, σ stretches them.

The whole algorithm in code

```
def scalar_loss(mu, log_sd, x_obs, response,
                noise_sd, S=64):
    q = Normal(mu, log_sd.exp())
    z = q.rsample((S,))                          # [S]
    pred = response(z)                           # [S]
    log_lik = Normal(pred[:, None], noise_sd) \
        .log_prob(x_obs).sum(-1)    # [S,N] -> [S]
    log_joint = log_lik + prior.log_prob(z)
    return -(log_joint - q.log_prob(z)).mean()

optimizer = Adam([mu, log_sd], lr=0.025)
for step in range(steps):
    optimizer.zero_grad()
    loss = scalar_loss(mu, log_sd, x_obs,
                       response, noise_sd)
    loss.backward()
    optimizer.step()
```

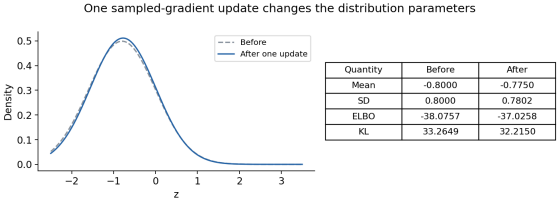
Notebook §4: scalar_loss and the training loop

Code	Pipeline step
`response, prior, likelihood`	Model
`Normal(mu, log_sd.exp())`	Family q_λ
`q.rsample((S,))`	Sample $z = \mu + \sigma\epsilon$
mean of log terms	Estimate $-\widehat{\mathcal{L}}$
`loss.backward()`	Gradient (autodiff)
`optimizer.step()`	Update λ

Shape check: z: $[S]$; log-likelihood: $[S, N]$, summed to $[S]$; mean over S: one scalar loss.

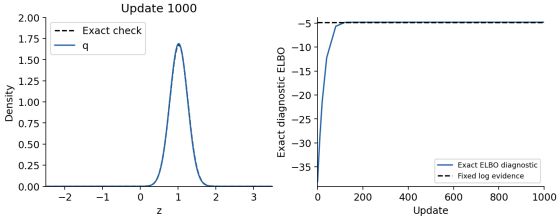
`rsample` (not `sample`) keeps z connected to μ and r. Minimizing $-\mathcal{L}$ is maximizing the ELBO.

Running it: one step, then many



Quantity	Before	After
Mean	-0.8000	-0.7750
SD	0.8000	0.7802
ELBO	-38.0757	-37.0258
KL	33.2649	32.2150

One noisy gradient step from $q = \mathcal{N}(-0.8, 0.8^2)$: Adam moves each parameter by $\eta = 0.025$.



After about 1000 steps q matches the exact posterior $\mathcal{N}(1.02, 0.24^2)$; the ELBO reaches $\log p(\mathbf{x})$.

A single stochastic step need not improve the exact ELBO; the trend over many steps does.

Pause and predict

Which quantities are optimizer parameters, and which are fresh random draws at every step?

Notebook §4–§5: one step, then the training animation

Use the samples: summaries and predictions

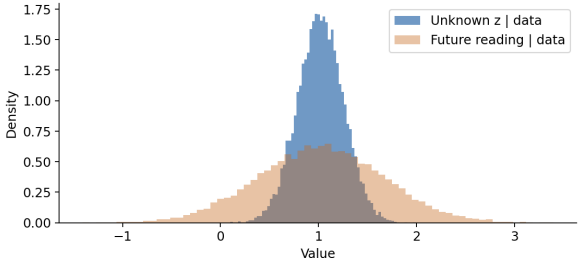
With $z^{(s)} \sim q_{\lambda^*}$, every question becomes an average:

$$\mathbb{E}[z \mid \mathbf{x}] \approx \tfrac{1}{S}\sum_s z^{(s)},$$
$$P(z > 1 \mid \mathbf{x}) \approx \tfrac{1}{S}\sum_s \mathbf{1}[z^{(s)} > 1],$$

95% interval: 2.5% and 97.5% sample quantiles.

Next reading: add the sensor noise to each sample,

$$x_*^{(s)} = z^{(s)} + \epsilon_*^{(s)}, \quad \epsilon_*^{(s)} \sim \mathcal{N}(0, \sigma_\epsilon^2).$$



Pause and predict

What does each interval describe: the fixed unknown z or a future noisy reading?

Notebook §5: posterior summaries and predictive

Check the result, not only the loss

Convergence (optimizer)

- Loss stable over steps?
- Same answer from repeated runs and seeds?
- Known-answer test recovered (as here)?

Accuracy (approximation and model)

- Can the family represent the posterior's shape?
- Do predictive checks match held-out data?
- Is the measurement model right?

A limit of the diagnostic

A higher ELBO means closer to the posterior *of this model*. It does not prove calibrated uncertainty or a correct model.

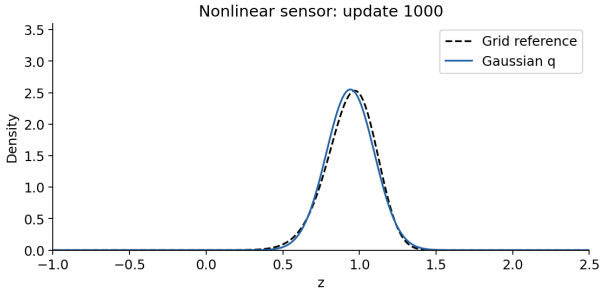
Notebook §5 and §10

Change the model, keep the code: a nonlinear sensor

$$h(z) = z + 0.2z^3, \qquad x_n \mid z \sim \mathcal{N}(h(z), \sigma_\epsilon^2).$$

Changes	the `response` function (one argument)
Same	prior, q, `rsample`, loss, optimizer, summaries

Prediction must use the new response: $x_*^{(s)} = h(z^{(s)}) + \epsilon_*^{(s)}$.



No closed-form posterior now; the grid is only a check.

Pause and predict

Which line of `scalar_loss` changes? How would you predict the next reading?

Notebook §6: nonlinear_response

Two unknowns: Bayesian linear regression

Data pairs $D = \{(x_n, y_n)\}$: input x_n, output y_n.

$$y_n = ax_n + b + \epsilon_n, \qquad \mathbf{w} = (a, b) \sim \mathcal{N}(\mathbf{0}, I).$$

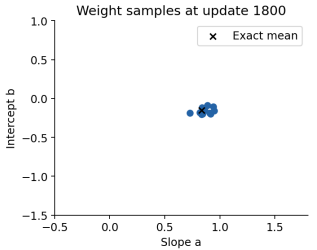
Mean-field family:

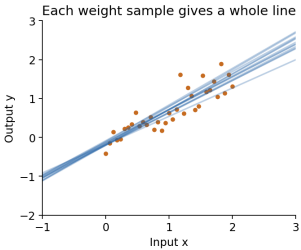
$$q(\mathbf{w}) = \mathcal{N}(a; \mu_a, \sigma_a^2)\,\mathcal{N}(b; \mu_b, \sigma_b^2),$$

so $\lambda = (\mu_a, r_a, \mu_b, r_b)$: four numbers.

Independence restricts q only; the true posterior can still be correlated.

Notebook §8: linear regression

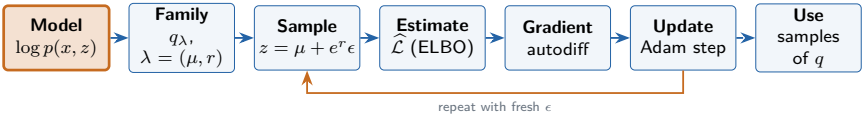




One draw of (a, b) = one whole line.

A Bayesian neural network: only the Model box changes

	Linear regression	Neural network
Forward function	$ax + b$	$f_{\mathbf{w}}(x) = \sum_{j=1}^{8} v_j \tanh(u_j x + b_j) + c$
Unknowns $\mathbf{w}$	2	$P = 8 + 8 + 8 + 1 = 25$ weights and biases
Learned λ	2 means, 2 log-SDs	25 means, 25 log-SDs (mean-field)
Sampling	$\mathbf{w} = \boldsymbol{\mu} + e^{\mathbf{r}} \odot \boldsymbol{\epsilon}, \;\; \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I)$	
Objective	expected NLL $+\mathrm{KL}(q\|p)$; $\mathrm{KL} = \frac{1}{2}\sum_j(\mu_j^2 + \sigma_j^2 - 1 - 2\log\sigma_j)$ (closed form)	



Notebook §9: bnn_forward

Two kinds of sampling, one training step

Random choice	Approximates	Size
$\mathbf{w}^{(s)} \sim q$	expectation over weights	S
batch B_t	sum over data	k of N

Minibatch loss (likelihood scaled, KL once):

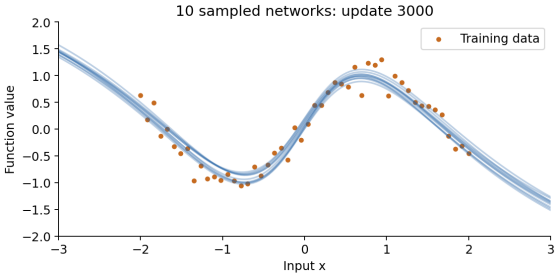
$$\widehat{\text{loss}} = \frac{N}{k}\frac{1}{S}\sum_{s}\sum_{n\in B_t} -\log p(y_n \mid x_n, \mathbf{w}^{(s)}) + \mathrm{KL}(q\|p).$$

```
optimizer.zero_grad()
w = mu + log_sd.exp()*torch.randn(S, P) # [S,P]
pred = bnn_forward(w, xb)               # [S,k]
nll = -Normal(pred, noise_sd).log_prob(yb)
loss = (N/k)*nll.sum(-1).mean() \
       + gaussian_kl(mu, log_sd)
loss.backward()     # grads for mu, log_sd
optimizer.step()
```

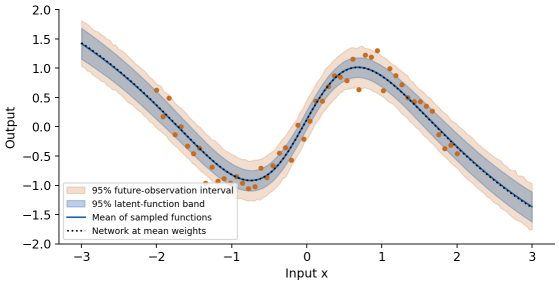
The same loop as the sensor; the optimizer owns 50 numbers, the sampled weights are intermediate values.

Notebook §7 and §9: minibatches and the BNN step

Using the trained BNN: average sampled networks



Each curve: one draw of all 25 parameters.



$$\mathrm{Var}(y_* \mid x_*, D) \approx \mathrm{Var}_q[f_{\mathbf{w}}(x_*)] + \sigma_\epsilon^2.$$

Predict by drawing $\mathbf{w}^{(s)} \sim q$, evaluating $f_{\mathbf{w}^{(s)}}(x_*)$, adding noise, then averaging or taking quantiles. $\mathbb{E}_q[f_{\mathbf{w}}(x_*)] \neq f_{\mathbb{E}_q[\mathbf{w}]}(x_*)$ in general; for classification, average probabilities, not logits.

Notebook §9: training animation and predictive intervals

The recipe for a new problem, and a check

Six questions (one per pipeline box)

1. What are the observations and unknowns?
2. Prior and likelihood: what is $\log p(x, z)$?
3. Which family q, and what can it miss?
4. How do samples enter the loss and gradients?
5. What does the optimizer update?
6. How do samples answer the question?

Notebook §10: transfer exercises with revealable answers

Check understanding

- Does the optimizer update z, the prior, or λ?
- The ELBO rises from -20 to -10. Has q improved?
- Why is the future-reading interval wider than the interval for z?
- If the likelihood changes, what code is reused?

References and implementation notes

- Blei, Kucukelbir and McAuliffe (2017). *Variational Inference: A Review for Statisticians*. https://arxiv.org/abs/1601.00670
- Kucukelbir et al. (2017). *Automatic Differentiation Variational Inference*. https://jmlr.org/papers/v18/16-107.html
- Blundell et al. (2015). *Weight Uncertainty in Neural Network*. https://proceedings.mlr.press/v37/blundell15.html
- PyTorch distributions: `rsample`, `log_prob`. https://docs.pytorch.org/docs/stable/distributions.html

The example fixes the prior and noise. More general models may need constrained transformations, richer families or other gradient estimators.

Acknowledgements/ Document preparation

This document was developed and converted into LaTeX slides and formatted with assistance from ChatGPT (OpenAI) and CoPilot (Microsoft). The instructor modified the source text and verified the final structure and wording.